Revision 24–27 · Benchmarking a server: baseline, sweeps, shapes & walls

Interactive revision of lesson 24, lesson 25, lesson 26 and lesson 27: one question at a time, logged in my own reasoning.

Q1 · Is a 70B FP8 baseline on two GPUs healthy? PARTIAL

Covers: 01, 04, 24

> **QUESTION**
>
> Llama 3 70B with FP8 weights (1 byte per parameter) runs on 2× H100 with tensor parallel 2, each GPU at 3.35 TB/s. A clean single-request baseline measures TPOT = 14 ms. Compute the TPOT floor and the MBU, and say whether decode is healthy.

How I solved it

1. Every decode step reads all the weights once: 70×10^9 parameters × 1 byte = **70 GB**.
2. With tensor parallel 2, each GPU holds half the weights and both read at the same time, so the bandwidth doubles: 2 × 3.35 TB/s = **6.7 TB/s**.
3. Floor = bytes ÷ bandwidth = $70 \times 10^9 \div 6.7 \times 10^{12} \approx 10.4 \times 10^{-3}$ s = **10.4 ms**.
4. MBU = floor ÷ measured = 10.4 ÷ 14 ≈ **74%**.
5. 74% sits inside the healthy 60–80% band, so decode is fine; nothing to investigate.

> **RE-DRILL**
>
> Redo the floor for the same model in 16-bit on 4 GPUs (140 GB ÷ 13.4 TB/s ≈ 10.4 ms: same floor, twice the GPUs).

Q2 · "32 users means 32× slower, right?" PARTIAL

Covers: 19, 25

> **QUESTION**
>
> A PM says serving 32 users at once instead of 1 makes each user 32× slower. Llama 3 8B in 16-bit (16 GB) on an H100 at 3.35 TB/s; each user holds ~1,000 tokens of KV at 128 KiB (≈ 128×10^3 bytes) per token. Estimate the decode step floor at 32 users, the real TPOT rise versus 4.8 ms, and the throughput gain.

How I solved it

1. Each step reads the weights once, shared by everyone: **16 GB**.

2. Each user's KV: 1,000 tokens × 128 × 10^3 B = **128 MB**, and every user's cache is read every step.

	Bytes per step	Step time
1 user	16 + 0.128 ≈ 16.1 GB	≈ 4.8 ms
32 users	16 + 32 × 0.128 = 16 + 4.1 ≈ 20.1 GB	20.1 × 10^9 ÷ 3.35 × 10^{12} ≈ **6.0 ms**

1. TPOT rise: 6.0 ÷ 4.8 ≈ **1.25×** (25% slower), not 32×.
2. Throughput: 32 ÷ 6.0 ms ≈ 5,300 tok/s versus 1 ÷ 4.8 ms ≈ 210 tok/s, about **25×**.
3. Answer to the PM: users share the weight read, so each extra user only adds their own KV bytes.

> **RE-DRILL**
>
> Redo it with 8,000-token contexts: the KV term becomes 32 × 1 GB = 32 GB, twice the weights, so the "free lunch" is gone.

Q3 · Pick the operating point from a sweep SKIPPED

Covers: 23, 25

> **QUESTION**
>
> A concurrency sweep gives tok/s and P99 TPOT at 1, 8, 16, 32, 64 and 128 users (150 → 5,000 tok/s; 6.5 → 26 ms). Every user must see at least 100 tokens/s at P99. Which concurrency do you run at, with what throughput, and what do you tell a colleague who wants "5,000 tok/s" in the capacity doc?

How I solved it

1. Per-user speed = 1,000 ÷ TPOT (ms), so "at least 100 tok/s" means P99 TPOT ≤ **10 ms**.

Concurrency	P99 TPOT	Per-user speed	Meets ≥ 100?
32	9.6 ms	≈ 104 tok/s	Yes
64	13.8 ms	≈ 72 tok/s	No
128	26 ms	≈ 38 tok/s	No

1. Run at the rightmost row under the line: **concurrency 32, 3,400 tok/s**.
2. 5,000 tok/s is the throughput peak, where each user gets ~38 tok/s: that's the wall, not capacity. 64 → 128 adds only 9% throughput but nearly doubles TPOT.

> **RE-DRILL**
>
> Re-drill: given a sweep table, convert the SLO into a TPOT line first, then scan for the last passing row.

Follow-up I asked: how can 32 users at ~104 tok/s each make 3,400 tok/s? Per-user speed is what one user sees; total throughput is the sum over everyone. 32 × ~104 ≈ 3,300, and the average user is a bit faster than the P99 user (3,400 ÷ 32 ≈ 106 tok/s).

Q4 · Why RAG traffic slows the chat users PARTIAL

Covers: 15, 16, 26

> **QUESTION**
>
> Chat (500-token prompts, 500-token answers) and RAG (6,000-token prompts, 300-token answers) share one H100 running Llama 3 8B. Since RAG was added, chat replies stream more slowly although chat traffic is unchanged. Give the two reasons.

Answer

- **Prefill interference (compute).** A 6,000-token prefill is a large burst of compute that runs inside the same scheduler steps as the chat users' decode, so chat tokens wait during it: longer gaps and stalls mid-stream. The chat requests aren't queued; they're already streaming and get paused.
- **Bigger caches reread every step (memory).** Each decode step reads every active user's whole KV cache. A RAG request holds ~6,300 tokens ≈ 0.8 GB versus ≈ 0.13 GB for chat, so each RAG user adds ~6× a chat user's bytes to every step and everyone's TPOT rises.
- Those big caches also fill the KV pool, so fewer requests fit, leading to queueing or preemption.
- Levers: cap the prefill chunk size, give each traffic type its own pool, or split prefill and decode onto different GPUs.

> **RE-DRILL**
>
> Re-drill: for any mixed workload, ask "what does it do to compute per step, and to bytes per step?"

Q5 · Silent tail spikes on an agent workload PARTIAL

Covers: 19, 26, 27

> **QUESTION**
>
> My server's `/metrics` shows a KV pool of 532,768 tokens, and `max_num_seqs` is vLLM's default of 1024 on this GPU. An agent product sends ~1,000-token prompts with ~6,000-token outputs. Under load, P99 TPOT spikes to several seconds every few minutes, with zero errors and no crash. What's happening, which counter confirms it, and what `max_num_seqs` would I set?

How I solved it

1. Size a request at the end, not at the start: 1,000 prompt + 6,000 output = **7,000 tokens**.
2. Requests that fit at final size: 532,768 ÷ 7,000 ≈ **76**.

3. vLLM admits up to 1024, more than 13× that. At admission each request holds only ~1,000 tokens, so ~530 get in; as outputs grow the pool hits 100%.
4. At 100% vLLM **preempts**: it evicts a running request, discards its cache and later recomputes prompt plus everything generated. No error, just a frozen request and a slower batch: the P99 spikes.
5. Confirm with `vllm:num_preemptions_total` climbing while `vllm:kv_cache_usage_perc` sits at 100%.
6. Fix: `--max-num-seqs` ≈ **70** (76, rounded down for headroom), so extra requests wait visibly in `num_requests_waiting` instead of thrashing.

> **RE-DRILL**
>
> Re-drill: each preemption re-prefills up to 7,000 tokens; estimate that cost with 32 μs per prompt token (≈ 0.22 s of GPU time).

Q6 · Code: find the three walls from a sweep PARTIAL

Covers: 25, 27

> **QUESTION**
>
> Write `find_walls(points, tpot_slo_ms, pool_tokens, tokens_per_request)` for a concurrency sweep of `(concurrency, tok_s, p99_tpot_ms)` rows that double each time. Return the saturation point (first row whose next doubling adds < 5%), the SLO crossing (interpolated in log2 concurrency, rounded down), the memory wall (pool ÷ final length), and the capacity (the smallest wall). Expected: SLO 10 ms, 2,000 tokens → `({'saturation': 128, 'slo': 34, 'memory': 266}, 'slo', 34)` ; SLO 50 ms, 7,000 tokens → `({'saturation': 128, 'slo': 242, 'memory': 76}, 'memory', 76)` .

How I solved it

1. **Saturation:** walk neighbouring rows and compute the fractional gain `tok_next / tok - 1` . 128 → 256 gives 5,150 ÷ 5,000 − 1 = 3% < 5%, so saturation is at **128**.
2. **SLO wall:** find the pair where this row passes and the next fails (9.6 ms at 32, 13.8 ms at 64 for a 10 ms target). The target sits (10 − 9.6) ÷ (13.8 − 9.6) ≈ 0.095 of the way between them. Because rows double, interpolate in log2: log2 c = 5 + 0.095 = 5.095, so c = 2^5.095 ≈ **34**.
3. **Memory:** 532,768 ÷ 2,000 = **266** requests fit at final length (76 at 7,000).
4. **Capacity:** the smallest wall wins: min(128, 34, 266) = 34, limited by the SLO. For the agent workload, min(128, 242, 76) = 76, limited by memory.

```python
def find_walls(points, tpot_slo_ms, pool_tokens, tokens_per_request):
    saturation = None
    for (c, tok, _), (c_next, tok_next, _) in zip(points, points[1:]):
        gain = tok_next / tok - 1
        if gain < 0.05:
            saturation = c
            break

    slo = None
    for (c, _, t), (c_next, _, t_next) in zip(points, points[1:]):
        if t <= tpot_slo_ms < t_next:
            frac = (tpot_slo_ms - t) / (t_next - t)
            log_c = math.log2(c) + frac * (math.log2(c_next) - math.log2(c))
            slo = math.floor(2**log_c)
            break

    memory = pool_tokens // tokens_per_request

    walls = {k: v for k, v in {"saturation": saturation, "slo": slo, "memory": memory}.items()
             if v is not None}
    limiting = min(walls, key=walls.get)
    return walls, limiting, walls[limiting]
```

RE-DRILL

Re-drill: add the TTFT SLO as a fourth wall, and handle a sweep that never fails the SLO (`slo` stays `None`).